COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.3 · Race Conditions

Mestre CyberSec · **Lesson 2.3 — Race Conditions** (4:58) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Race condition: dois processos tentam MUDAR O MESMO recurso ao mesmo tempo — o resultado depende da ordem. TOCTOU (Time-of-Check to Time-of-Use): checa permissão em t1, executa em t2 — gap exploitação. Defesa: operação ATÔMICA, lock.

💡 Key Exam Highlights & Critical Concepts

- Race condition: resultado depende da ordem de execução
- TOCTOU: gap entre checar permissão e usar o recurso

🎯 Exam Tips & Typical CompTIA Scenarios

- 'checks permission at time A but executes at time B' = TOCTOU

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.3 — Race Conditions)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• Race Condition (N/A)

A vulnerability where multiple processes or threads access and modify the same resource concurrently, so the security outcome depends on the uncontrolled order of execution.

⚠️ ***Exam Trap:*** *CompTIA costuma descrever 'result depends on execution order' ou 'same resource changed at the same time'; não confunda com falta de atualização ou erro de permissão isolado.*

• Time-of-Check to Time-of-Use (TOCTOU)

A race condition where a system checks a condition or permission at one time, but the resource changes before it is used, allowing the attacker to exploit the gap.

⚠️ ***Exam Trap:*** *Gatilho clássico: 'checks permission at time A but executes at time B' = TOCTOU. Palavra-chave: gap entre check e use.*

• Atomic Operation (N/A)

An operation that executes completely or not at all, with no intermediate state visible to other processes, preventing race-condition interference.

⚠️ ***Exam Trap:*** *Defesa-chave em race conditions: se a questão pede impedir mudança entre check e use, pense em operação atômica ou lock, não em criptografia.*

• Lock (N/A)

A synchronization mechanism that allows only one process or thread to access a shared resource at a time, preventing concurrent modification.

⚠️ ***Exam Trap:*** *Lock é mitigação direta de race condition; CompTIA pode citar mutex ou lock para garantir acesso exclusivo a recurso compartilhado.*

• Mutual Exclusion (Mutex) (MUTEX)

A locking mechanism that ensures only one thread can enter a critical section or access a shared resource at a time.

⚠️ ***Exam Trap:*** *Mutex = lock de exclusão mútua. Não confunda com semáforo, que pode permitir múltiplos acessos limitados, nem com criptografia.*

• **Critical Section (N/A)**
The portion of code that accesses a shared resource and must be protected so only one process or thread executes it at a time.

⚠️ ***Exam Trap:*** *Se a questão menciona código que altera recurso compartilhado, a proteção da seção crítica com lock ou atomicidade evita race condition.*

• **Concurrency (N/A)**
The execution of multiple processes or threads that can overlap in time and compete for shared resources, creating race-condition risk.

⚠️ ***Exam Trap:*** *Concorrência é o cenário que possibilita race condition; por si só não é a vulnerabilidade, mas a falta de sincronização é.*

• **Non-Atomic Operation (N/A)**
A multi-step operation that can be interrupted between steps, allowing another process to change the resource and cause a race condition or TOCTOU flaw.

⚠️ ***Exam Trap:*** *TOCTOU geralmente envolve uma sequência não atômica de checar e depois usar; a correção é tornar a operação atômica ou protegida por lock.*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 TOCTOU (Time-of-Check to Time-of-Use)

🏦 **Real-World Analogy:** You withdraw $100 from an ATM. The bank checks your balance at 12:00:00 ($100 available). Between the balance check and cash dispensing (12:00:01), you trigger a simultaneous app withdrawal. If the account isn't locked during that split second, you get $200 with only $100.

🔍 **Root Cause & Background:** Multiple processes or threads access shared resources concurrently without mutual exclusion locks (Mutexes).

🛡️ **GRC Auditor Perspective:** Enforce concurrency code reviews and mandate atomic database transactions with row-level locks for privileged operations.

⚠️ **How CompTIA Tests This:** Trigger: 'vulnerability occurring between checking a permission and performing the action' -> Race Condition / TOCTOU.

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Two concurrent events race to access shared resource causing inconsistent state	**Race Condition**
Attacker modifies a file between permission validation and actual execution	**TOCTOU**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A developer at a fintech startup writes a script that checks a user's account balance before processing a withdrawal. The check and the withdrawal are two separate database calls. An attacker discovers they can submit two simultaneous withdrawal requests for the full balance, and both transactions are approved because the balance check happens before either withdrawal is recorded. Which vulnerability BEST describes this issue?

A) Time-of-Check to Time-of-Use (TOCTOU) race condition

B) SQL injection through unsanitized input

C) Cross-site request forgery exploiting session cookies

D) Insufficient input validation on the withdrawal amount

QUESTION 2 · EXAM SCENARIO

A web application lets users redeem a single-use discount code. An attacker uses a script to fire 50 HTTP requests at the same moment when the promotion goes live. Despite the code being valid only once, the attacker successfully applies the discount on five separate orders. The root cause is that the 'is code already used?' check and the 'mark code as used' write happen as two independent queries. Which mitigation should the developer implement?

A) Combine the validation and redemption into a single atomic database transaction with row-level locking

B) Add CAPTCHA to the checkout form to slow down automated requests

C) Hash the discount code before storing it in the database

D) Enforce HTTPS on the checkout endpoint to encrypt the payload

QUESTION 3 · EXAM SCENARIO

A Linux server runs a privileged cron job that does the following: it checks whether the file /tmp/payload.sh is owned by root, and if so, runs it. A local attacker replaces /tmp/payload.sh with a malicious script in the tiny window between the ownership check and the execution. Which attack is this, and what is the BEST fix?

A) TOCTOU attack; fix by making the check-and-execute a single atomic operation using a file descriptor (open + fstat + fexecve) instead of separate stat() and execve() calls

B) Buffer overflow attack; fix by enabling ASLR and stack canaries

C) Symlink race attack; fix by disabling /tmp execution with a read-only mount

D) Privilege escalation via sudo misconfiguration; fix by editing the sudoers file

QUESTION 4 · EXAM SCENARIO

An authentication library allows users to change their password. The flow is: (1) verify the old password matches the stored hash, (2) if it matches, overwrite the stored hash with the new one. A pentester demonstrates that if an attacker submits the password-change request at the exact moment a victim's legitimate change-password request is in flight, the attacker can hijack the account. What type of concurrency flaw is being exploited?

A) Race condition in a non-atomic credential-update operation, exploitable between the verify-old and write-new steps

B) Man-in-the-middle attack due to missing TLS on the password-change endpoint

C) Replay attack against the session cookie used by the change-password API

D) Weak password hashing algorithm susceptible to offline cracking

QUESTION 5 · EXAM SCENARIO

A security review of a multi-threaded C++ daemon finds the following pattern in the access-control module: 'if (user.role == admin) { sleep(2); runAdminCommand(); }'. Two threads evaluating the same user object concurrently can both observe role == admin before either thread reaches runAdminCommand(). Which secure-coding construct should the developer apply to the entire critical section?

A) Acquire a mutex (mutual exclusion lock) around the check-and-execute critical section so that only one thread at a time can evaluate the privilege and act on it

B) Increase the thread pool size to handle more concurrent users

C) Disable threading entirely by setting a single-threaded build flag

D) Replace the role check with a regex match on the username string

QUESTION 6 · EXAM SCENARIO

An e-commerce platform's API checks whether a product is in stock by reading from a cache, then decrements the inventory count in the database. During a flash sale, customers report that items showing 'in stock' are sold multiple times, leading to overselling. The cache lookup and the database decrement are non-atomic. Which control BEST eliminates this overselling race condition?

A) Use a database-level atomic decrement (e.g., UPDATE stock SET qty = qty - 1 WHERE product_id = ? AND qty > 0) inside a locked transaction so the read and write are inseparable

B) Serve static marketing pages from a CDN to reduce backend load during the sale

C) Implement HTTP cookie-based session affinity to route users to the same backend node

D) Switch from REST to GraphQL to optimize query payloads

QUESTION 7 · EXAM SCENARIO

A security analyst is reviewing the source code of a session-management library and finds the pattern: 'if (session.isValid()) { session.regenerateId(); performAction(); }'. An attacker times their calls so that two requests share the same valid session ID at the moment of regeneration, allowing one request to act under the victim's identity. Which term BEST classifies this flaw?

A) Time-of-Check to Time-of-Use race condition on session validity, allowing session fixation or identity confusion

B) Insecure direct object reference (IDOR) due to predictable session identifiers

C) Cross-site scripting via unsanitized session data in error pages

D) Improper error handling leaking stack traces to the browser

QUESTION 8 · EXAM SCENARIO

A REST API implements rate limiting by reading a counter from a file, checking if it exceeds 100, and if not, incrementing the counter and processing the request. An attacker sends 200 requests in parallel and all 200 are processed even though the limit is 100. The fix the team proposes is to read and increment the counter in one indivisible step. What secure concept does this fix correspond to?

A) Atomic operation: combining the read and increment into a single indivisible step so concurrent requests cannot interleave between check and modify

B) Symmetric encryption of the counter file at rest using AES-256

C) Role-based access control on the API endpoint enforcing read-only for guests

D) Public key infrastructure with X.509 mutual TLS between client and server

QUESTION 9 · EXAM SCENARIO

A cloud function automatically provisions a virtual machine by: (1) calling the IAM API to verify the caller has 'compute.create' permission, (2) then calling the cloud API to actually create the VM. A tester shows that revoking the caller's permission between steps 1 and 2 still results in a VM being created because the cached decision from step 1 is reused. Which control MOST directly addresses this Time-of-Check to Time-of-Use flaw?

A) Re-check authorization at the moment of resource creation and require the access decision to be bound to a short-lived, single-use signed token obtained inside the same transaction

B) Rotate the cloud provider's root API keys every 24 hours

C) Enable VPC flow logs on the project to capture all API calls for forensic review

D) Store IAM policies in a public-read Cloud Storage bucket for transparency

QUESTION 10 · EXAM SCENARIO

A kernel developer is patching a vulnerable driver that calls copy_from_user() to read a userspace buffer, then uses the buffer contents in a privileged operation. A security advisory notes that the size argument passed to copy_from_user() is read from a userspace variable that another thread could modify concurrently. Which mitigation eliminates the race between size validation and use?

A) Snapshot the size into a kernel-local variable, validate it once, and pass that validated local copy to copy_from_user() so the value cannot change between check and use

B) Disable preemptive scheduling in the kernel configuration to prevent thread swaps

C) Encrypt userspace memory using a kernel-internal AES key before copying

D) Move the buffer copy logic into userspace using a syscall wrapper

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option A — Time-of-Check to Time-of-Use (TOCTOU) race condition

The flaw is a classic TOCTOU race condition: the system checks the balance at time t1 and uses (debits) it at time t2. Because they are separate, non-atomic operations, an attacker can interleave requests so both checks pass before either debit occurs. SQL injection, CSRF, and input validation are unrelated to the temporal gap between check and use. Atomic operations (or row-level locking) must wrap the entire balance-check-and-debit sequence so the two operations cannot be split.

Question 2: Option A — Combine the validation and redemption into a single atomic database transaction with row-level locking

Atomic transactions with row-level locking (e.g., SELECT ... FOR UPDATE) ensure the 'check used?' and 'mark used?' steps cannot be split across concurrent requests, eliminating the TOCTOU window. CAPTCHA only slows bots and does not fix the underlying logic flaw. Hashing addresses confidentiality, not race conditions. HTTPS protects data in transit but does not prevent concurrent logic abuse.

Question 3: Option A — TOCTOU attack; fix by making the check-and-execute a single atomic operation using a file descriptor (open + fstat + fexecve) instead of separate stat() and execve() calls

The script performs stat() at time t1 and execve() at time t2. An attacker can swap the file in between, classic TOCTOU. Using open() to obtain a file descriptor, then fstat() and fexecve() on that fd makes the operation refer to the same inode atomically. While disabling /tmp execution partially mitigates, it is environment-specific and brittle. Buffer overflow and sudoers issues are unrelated categories.

Question 4: Option A — Race condition in a non-atomic credential-update operation, exploitable between the verify-old and write-new steps

The 'verify old hash' and 'write new hash' are non-atomic steps. Concurrent requests can race so an attacker slips in during the gap, leveraging a TOCTOU-style race on shared credential state. Mitigations include wrapping both steps in a database transaction with the row locked, or generating a per-request nonce/ticket bound to the verified old credential. MITM requires TLS absence, replay needs stolen cookies, and weak hashing is offline-cracking — none address concurrent logic gaps.

Question 5: Option A — Acquire a mutex (mutual exclusion lock) around the check-and-execute critical section so that only one thread at a time can evaluate the privilege and act on it

A mutex around the critical section guarantees mutual exclusion: only one thread can hold the lock, perform the role check, and execute the privileged command atomically — closing the TOCTOU window. Increasing thread pool size makes concurrency worse. Disabling threading is impractical for production daemons. Regex on usernames is a logic/input issue, not a concurrency one.

Question 6: Option A — Use a database-level atomic decrement (e.g., UPDATE stock SET qty = qty - 1 WHERE product_id = ? AND qty > 0) inside a locked transaction so the read and write are inseparable

A single atomic UPDATE with a WHERE qty > 0 predicate performs the check-and-decrement as one indivisible operation. The database engine serializes contending updates, so two concurrent requests cannot both pass the 'in stock?' check and oversell. CDN offload, sticky sessions, and GraphQL payload changes address performance or routing — they do not close the logical TOCTOU gap between cache-read and DB-write.

Question 7: Option A — Time-of-Check to Time-of-Use race condition on session validity, allowing session fixation or identity confusion

The session validity is checked at t1 and the action is performed at t2 using a state that can change between those points — a textbook TOCTOU race. Atomic session regeneration (e.g., binding the new ID to the request nonce inside a locked transaction) closes the window. IDOR concerns object references, XSS concerns output encoding, and stack-trace leaks concern error handling — none of these match the temporal-check-to-use flaw.

Question 8: Option A — Atomic operation: combining the read and increment into a single indivisible step so concurrent requests cannot interleave between check and modify

Atomic operations (or compare-and-swap / lock + read + write) eliminate the TOCTOU window by making the entire check-and-modify sequence uninterruptible to other threads or processes. The counter is a shared resource; non-atomic access lets racers bypass the limit. AES encryption, RBAC, and mTLS address confidentiality, authorization, and transport authentication respectively — they do not fix concurrency flaws.

Question 9: Option A — Re-check authorization at the moment of resource creation and require the access decision to be bound to a short-lived, single-use signed token obtained inside the same transaction

The TOCTOU gap is between IAM check and compute create. Re-validating authorization atomically at use time — typically by minting a short-lived capability/token inside the checked transaction and redeeming it at creation — eliminates the gap. Long-lived API key rotation, VPC logs, and public IAM storage do not address concurrent authorization drift and actually weaken security posture in some cases.

Question 10: Option A — Snapshot the size into a kernel-local variable, validate it once, and pass that validated local copy to copy_from_user() so the value cannot change between check and use

Taking a local copy of the size and validating it before passing it to copy_from_user() turns the previously non-atomic check-and-use into an atomic operation on kernel-controlled memory, eliminating TOCTOU. Disabling preemption is fragile and breaks real-time guarantees. Encrypting userspace memory does not prevent a racing thread from mutating the size variable, and moving the logic to userspace just expands the attack surface.